

Final Project: Regression and Classification with Neural Networks

Enock Owusu (22688016)

Part 0: Data Preparation and Exploratory Analysis

This section of the project looks at regression and classification tasks using multiple neural network architectures including DNN, CNN, LSTM, and hybrid models. Also data preparation and exploratory analysis was also conducted.

2.1 Jena Climate Dataset

```
'data.frame':  420551 obs. of  15 variables:
 $ Date.Time      : chr  "01.01.2009 00:10:00" "01.01.2009 00:20:00" "01.01.2009 00:30:00" "
 $ p..mbar.       : num  997 997 997 997 997 ...
 $ T..degC.       : num  -8.02 -8.41 -8.51 -8.31 -8.27 -8.05 -7.62 -7.62 -
7.91 -8.43 ...
 $ Tpot..K.       : num  265 265 265 265 265 ...
 $ Tdew..degC.    : num  -8.9 -9.28 -9.31 -9.07 -9.04 -8.78 -8.3 -8.36 -
8.73 -9.34 ...
 $ rh....        : num  93.3 93.4 93.9 94.2 94.1 94.4 94.8 94.4 93.8 93.1 ...
 $ VPmax..mbar.   : num  3.33 3.23 3.21 3.26 3.27 3.33 3.44 3.44 3.36 3.23 ...
 $ VPact..mbar.   : num  3.11 3.02 3.01 3.07 3.08 3.14 3.26 3.25 3.15 3 ...
 $ VPdef..mbar.   : num  0.22 0.21 0.2 0.19 0.19 0.19 0.18 0.19 0.21 0.22 ...
 $ sh..g.kg.      : num  1.94 1.89 1.88 1.92 1.92 1.96 2.04 2.03 1.97 1.88 ...
 $ H2OC..mmol.mol.: num  3.12 3.03 3.02 3.08 3.09 3.15 3.27 3.26 3.16 3.02 ...
 $ rho..g.m..3.   : num  1308 1310 1310 1309 1309 ...
 $ wv..m.s.       : num  1.03 0.72 0.19 0.34 0.32 0.21 0.18 0.19 0.28 0.59 ...
 $ max..wv..m.s.  : num  1.75 1.5 0.63 0.5 0.63 0.63 0.63 0.5 0.75 0.88 ...
 $ wd..deg.       : num  152 136 172 198 214 ...
```

	Date.Time	p..mbar.	T..degC.	Tpot..K.	Tdew..degC.	rh....
1	01.01.2009 00:10:00	996.52	-8.02	265.40	-8.90	93.3
2	01.01.2009 00:20:00	996.57	-8.41	265.01	-9.28	93.4
3	01.01.2009 00:30:00	996.53	-8.51	264.91	-9.31	93.9
4	01.01.2009 00:40:00	996.51	-8.31	265.12	-9.07	94.2
5	01.01.2009 00:50:00	996.51	-8.27	265.15	-9.04	94.1
6	01.01.2009 01:00:00	996.50	-8.05	265.38	-8.78	94.4

	VPmax..mbar.	VPact..mbar.	VPdef..mbar.	sh..g.kg.	H2OC..mmol.mol.	rho..g.m..3.
1	3.33	3.11	0.22	1.94	3.12	1307.75
2	3.23	3.02	0.21	1.89	3.03	1309.80
3	3.21	3.01	0.20	1.88	3.02	1310.24
4	3.26	3.07	0.19	1.92	3.08	1309.19
5	3.27	3.08	0.19	1.92	3.09	1309.00
6	3.33	3.14	0.19	1.96	3.15	1307.86

	wv..m.s.	max..wv..m.s.	wd..deg.
1	1.03	1.75	152.3
2	0.72	1.50	136.1
3	0.19	0.63	171.6
4	0.34	0.50	198.0
5	0.32	0.63	214.3
6	0.21	0.63	192.7

[1] 420551

[1] 70041

A tibble: 6 x 15

	Date.Hour	p..mbar.	T..degC.	Tpot..K.	Tdew..degC.	rh....	VPmax..mbar.
	<dtm>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2009-01-01 00:00:00	997.	-8.30	265.	-9.12	93.8	3.26
2	2009-01-01 01:00:00	997.	-8.06	265.	-8.86	93.9	3.32
3	2009-01-01 02:00:00	997.	-8.76	265.	-9.61	93.5	3.14
4	2009-01-01 03:00:00	997.	-8.90	264.	-9.79	93.2	3.11
5	2009-01-01 04:00:00	997.	-9.35	264.	-10.3	92.4	3.00
6	2009-01-01 05:00:00	998.	-9.72	264.	-10.7	92.4	2.92

i 8 more variables: VPact..mbar. <dbl>, VPdef..mbar. <dbl>, sh..g.kg. <dbl>,
H2OC..mmol.mol. <dbl>, rho..g.m..3. <dbl>, wv..m.s. <dbl>,
max..wv..m.s. <dbl>, wd..deg. <dbl>

tibble [70,041 x 15] (S3: tbl_df/tbl/data.frame)

\$ Date.Hour : POSIXct[1:70041], format: "2009-01-01 00:00:00" "2009-01-01 01:00:00" ...

```

$ p..mbar.      : num [1:70041] 997 997 997 997 997 ...
$ T..degC.      : num [1:70041] -8.3 -8.06 -8.76 -8.9 -9.35 ...
$ Tpot..K.      : num [1:70041] 265 265 265 264 264 ...
$ Tdew..degC.   : num [1:70041] -9.12 -8.86 -9.61 -9.79 -10.34 ...
$ rh....       : num [1:70041] 93.8 93.9 93.5 93.2 92.4 ...
$ VPmax..mbar.  : num [1:70041] 3.26 3.32 3.14 3.11 3 ...
$ VPact..mbar.  : num [1:70041] 3.06 3.12 2.94 2.9 2.77 ...
$ VPdef..mbar.  : num [1:70041] 0.202 0.202 0.202 0.21 0.232 ...
$ sh..g.kg.     : num [1:70041] 1.91 1.95 1.84 1.81 1.73 ...
$ H2OC..mmol.mol.: num [1:70041] 3.07 3.13 2.95 2.91 2.78 ...
$ rho..g.m..3.  : num [1:70041] 1309 1308 1312 1313 1315 ...
$ wv..m.s.      : num [1:70041] 0.52 0.317 0.248 0.177 0.29 ...
$ max..wv..m.s. : num [1:70041] 1.002 0.712 0.607 0.607 0.67 ...
$ wd..deg.      : num [1:70041] 174 172 197 157 150 ...

```

Brief Description

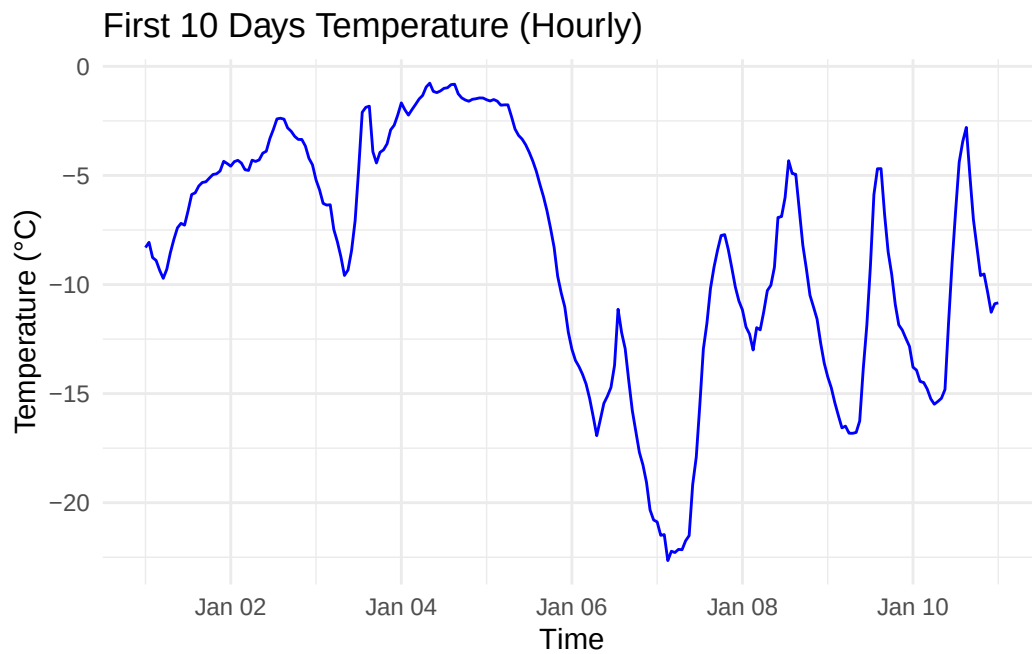
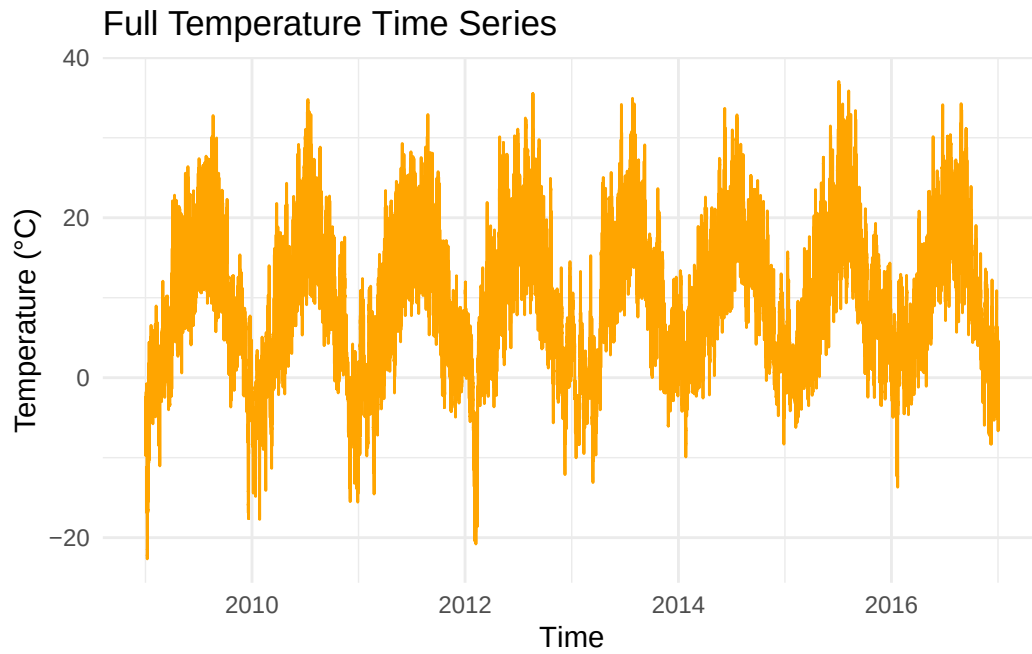
The Jena Climate dataset was imported from the given CSV file, and then the Date Time column was properly transformed to a datetime format using the `lubridate` library. The original dataset contained measurements made at every ten minutes. In order to adapt the data for the modelling purposes, the resampling of the data has been done by changing the observation frequency from 10 minutes to one hour where the average value of all numeric variables for the respective hour was calculated. This step ensured that each row would represent an hour since a 120-hour historical period was used to train neural network models further on. There were 420551 observations before performing the resampling procedure; however, after resampling there remained 70041 observations only.

Date.Hour	p..mbar.	T..degC.	Tpot..K.	Tdew..degC.
0	0	0	0	0
rh....	VPmax..mbar.	VPact..mbar.	VPdef..mbar.	sh..g.kg.
0	0	0	0	0
H2OC..mmol.mol.	rho..g.m..3.	wv..m.s.	max..wv..m.s.	wd..deg.
0	0	0	0	0

Missing Values Analysis

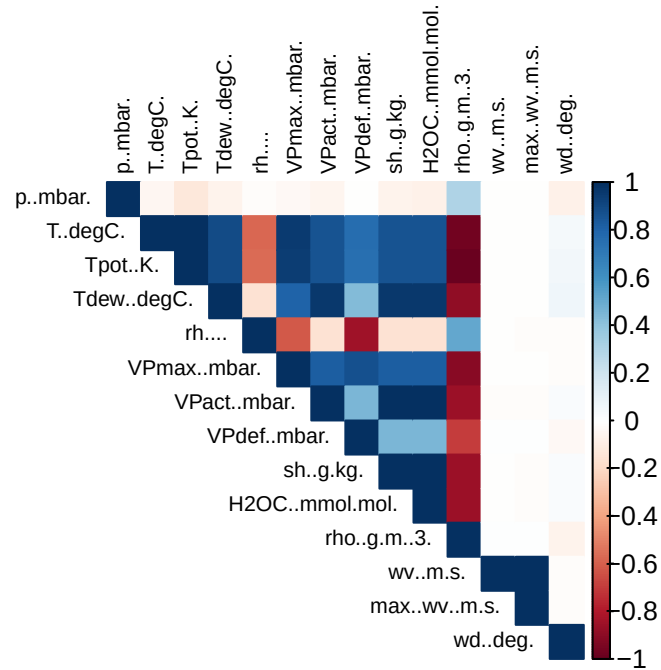
Missing values in the dataset were tested by finding the total number of missing observations per variable in the dataset. It can be seen from the results that no missing values of significance exist within all the variables after the resampling process done on an hourly basis. In this case, due to the resampling process being performed with mean aggregation and `na.rm` set

to true, any missing value existing in the dataset was addressed appropriately through the mean aggregation process. This was due to the fact that introducing missing value imputation techniques could have introduced a biased effect within the data.



Time Series Visualization

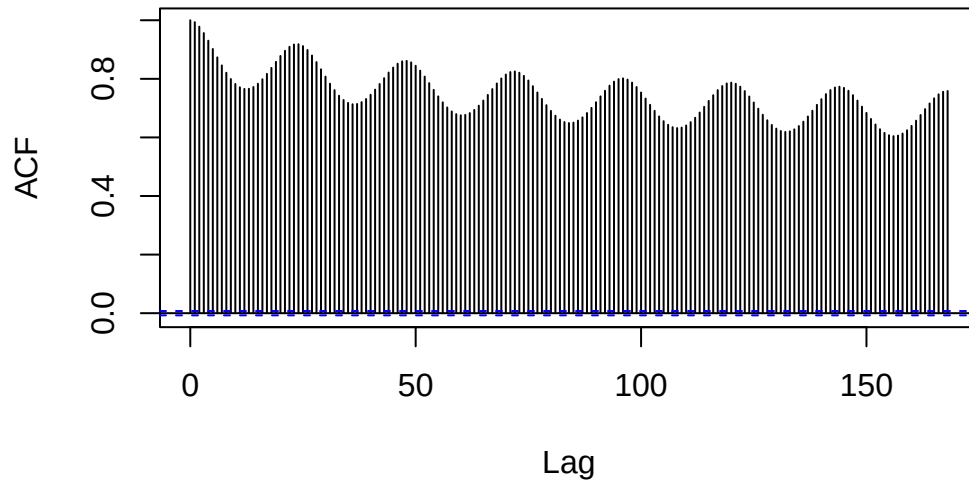
The complete temperature data shows definite trends in terms of seasons. There is a periodic rise and fall in temperatures every year, which clearly indicates that there is an underlying seasonality in temperature trends. This is evident in the graph plotted for the first ten days' temperatures measured per hour, which shows a definite diurnal trend. This trend shows a definite increase in temperatures during the day and fall during the night. This shows that there is significant variation within the day as well. Seasonality and diurnal trends make it essential to choose 120 hours as the look-back period.



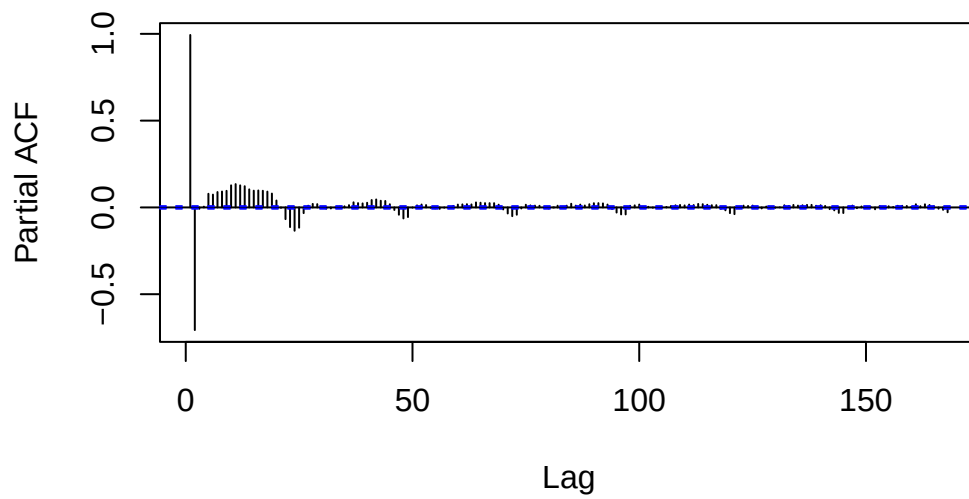
Correlation Analysis and Multicollinearity

In the correlation matrix, the relationship between all predictor variables included in the data set is highlighted. Many variables demonstrate a high positive or negative correlation, especially when comparing variables related to temperature, such as `T..degC.`, `Tpot..K.`, `Tdew..degC.`, and those related to vapor pressure. In turn, this suggests that there is a problem of multicollinearity in which predictors provide redundant data. Although multicollinearity does not pose a significant problem for neural networks, it is crucial to understand how these correlations occur.

ACF of Hourly Temperature Series

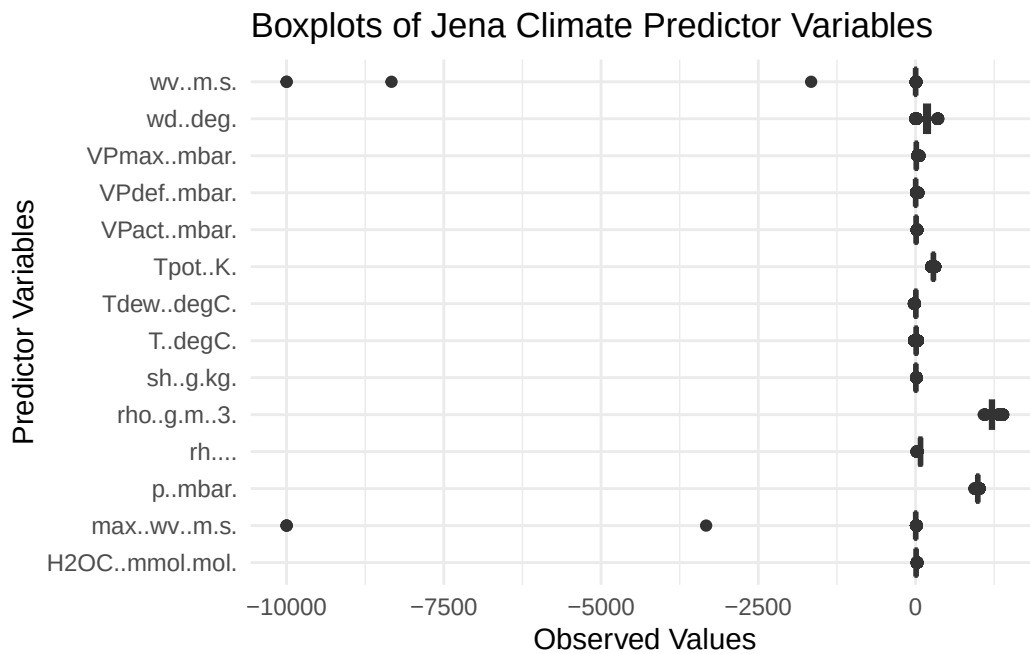


PACF of Hourly Temperature Series



ACF and PACF Interpretation

The ACF plot shows that temperature values are highly correlated across many previous hours. The slow decay in the autocorrelation function suggests strong temporal dependence in the series. This is expected because current temperature is influenced by recent past temperatures. The PACF plot also shows important partial correlations at earlier lags, indicating that past hourly observations contain useful predictive information. The use of a 120-hour lookback window is therefore appropriate because it allows the model to use five days of historical weather information to predict temperature 24 hours ahead. This window is long enough to capture short-term hourly variation, daily cycles, and broader multi-day temperature patterns.



Predictor Range and Need for Normalization

The box plots indicate that the predictor variables are highly diverse in terms of their scales of measurement. Some examples of such predictor variables include those related to atmospheric pressure, temperature, humidity, wind direction, and vapor pressure, among others, which differ from each other in terms of their units and numerical scale. This validates the necessity of performing normalization before applying the neural network model. The reason for doing so is that neural networks are affected by the magnitude of the predictor variables.

p. .mbar.	T. .degC.	Tpot. .K.	Tdew. .degC.
Min. : -6.57247	Min. : -3.73229	Min. : -3.7544	Min. : -4.24662

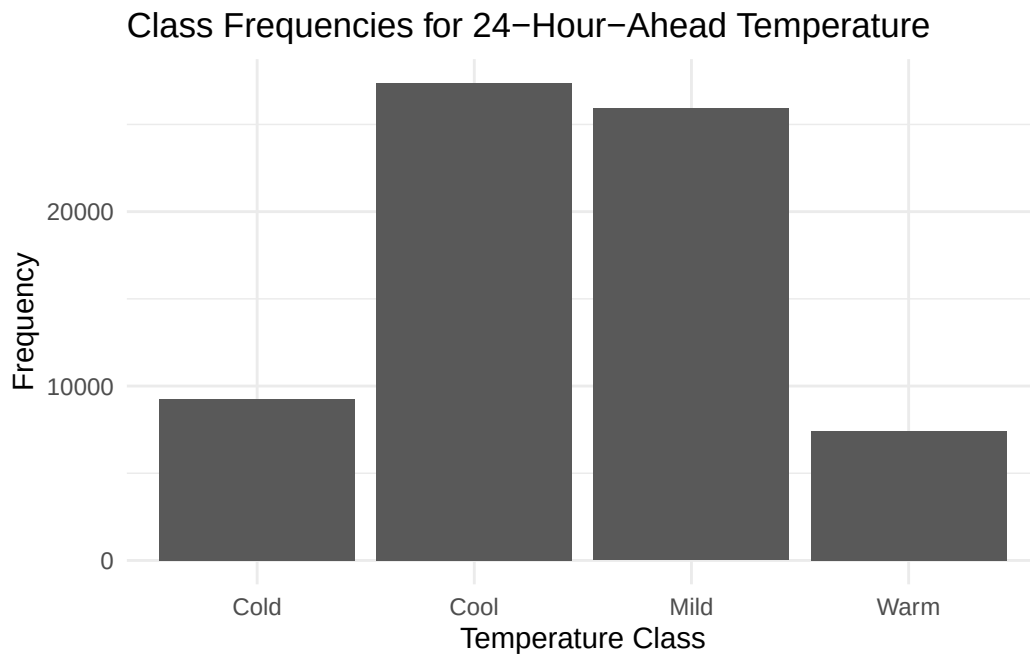
1st Qu.: -0.60791	1st Qu.: -0.71130	1st Qu.: -0.6980	1st Qu.: -0.67247
Median : 0.05157	Median : 0.01673	Median : 0.0201	Median : 0.06105
Mean : 0.00000	Mean : 0.00000	Mean : 0.0000	Mean : 0.00000
3rd Qu.: 0.65303	3rd Qu.: 0.71701	3rd Qu.: 0.7103	3rd Qu.: 0.75844
Max. : 2.93737	Max. : 3.09010	Max. : 3.0771	Max. : 2.63597
rh....	VPmax..mbar.	VPact..mbar.	VPdef..mbar.
Min. : -3.8361	Min. : -1.6367	Min. : -2.0573	Min. : -0.8284
1st Qu.: -0.6474	1st Qu.: -0.7561	1st Qu.: -0.7844	1st Qu.: -0.6539
Median : 0.2022	Median : -0.2134	Median : -0.1441	Median : -0.3820
Mean : 0.0000	Mean : 0.0000	Mean : 0.0000	Mean : 0.0000
3rd Qu.: 0.8094	3rd Qu.: 0.5347	3rd Qu.: 0.6809	3rd Qu.: 0.2639
Max. : 1.4400	Max. : 5.8945	Max. : 4.4569	Max. : 8.0527
sh..g.kg.	H2OC..mmol.mol.	rho..g.m..3.	wv..m.s.
Min. : -2.0493	Min. : -2.0567	Min. : -3.27470	Min. : -1.4719
1st Qu.: -0.7827	1st Qu.: -0.7828	1st Qu.: -0.70973	1st Qu.: -0.7481
Median : -0.1422	Median : -0.1407	Median : -0.08129	Median : -0.2373
Mean : 0.0000	Mean : 0.0000	Mean : 0.00000	Mean : 0.0000
3rd Qu.: 0.6772	3rd Qu.: 0.6786	3rd Qu.: 0.64999	3rd Qu.: 0.4819
Max. : 4.5215	Max. : 4.4933	Max. : 4.34120	Max. : 7.3890
max..wv..m.s.	wd..deg.		
Min. : -1.6051	Min. : -2.3497		
1st Qu.: -0.7688	1st Qu.: -0.4690		
Median : -0.2244	Median : 0.2402		
Mean : 0.0000	Mean : 0.0000		
3rd Qu.: 0.5332	3rd Qu.: 0.6797		
Max. : 7.4754	Max. : 2.4683		

Z-score Normalization

Z-score normalization was applied to all numeric variables using only the mean and standard deviation computed from the training set. The validation and test sets were then transformed using the same training-set statistics. This is important because validation and test data must represent unseen future observations. If their means and standard deviations are used during scaling, information from the future would leak into the training process. Such data leakage can produce overly optimistic validation and test performance because the model indirectly benefits from information that would not be available in real forecasting conditions.

```
# A tibble: 4 x 3
  temp_class    n percentage
  <chr>      <int>      <dbl>
1 Cold       9274        13.2
```


2 Cool	27376	39.1
3 Mild	25918	37.0
4 Warm	7449	10.6



A Derived Temperature Classification Label

A four-class classification target was created from the 24-hour-ahead temperature. Temperatures below 0°C were classified as Cold, temperatures from 0°C to 9.9°C as Cool, temperatures from 10°C to 19.9°C as Mild, and temperatures of 20°C or above as Warm. The class frequency table shows the distribution of the four temperature categories. If some classes occur more frequently than others, then the classification problem is class-imbalanced. This is important because a model may achieve high accuracy by favouring the dominant class while performing poorly on minority classes. Therefore, macro-F1 and the confusion matrix are necessary evaluation metrics in addition to accuracy.

```
[1] 128 120 14
```

```
NULL
```

Data Generator Function

A custom data generator function was developed to construct input-output sequences suitable for neural network training. The function accepts parameters including lookback (120 hours), delay (24 hours), step size, batch size, shuffle option, and task type (regression or classification). For each sample, the generator extracts a sequence of past observations over a 120-hour window and pairs it with a target value corresponding to the temperature 24 hours ahead. This sliding window approach transforms the time-series data into a supervised learning format. For regression tasks, the target is the continuous temperature value. For classification tasks, the target is the categorical temperature class (Cold, Cool, Mild, Warm), encoded appropriately. The generated input data has a three-dimensional structure: (batch size, time steps, number of features), which is suitable for deep learning architectures such as DNN, CNN, and LSTM. Verification of the output shapes confirms that the generator produces correctly structured batches for model training. This approach ensures memory efficiency and scalability, as data is generated in batches rather than loaded entirely into memory, which is particularly important for large time-series datasets.

```
[1] 60000    28    28
```

```
[1] 10000    28    28
```

```
[1]    0 255
```

```
[1]    0 255
```

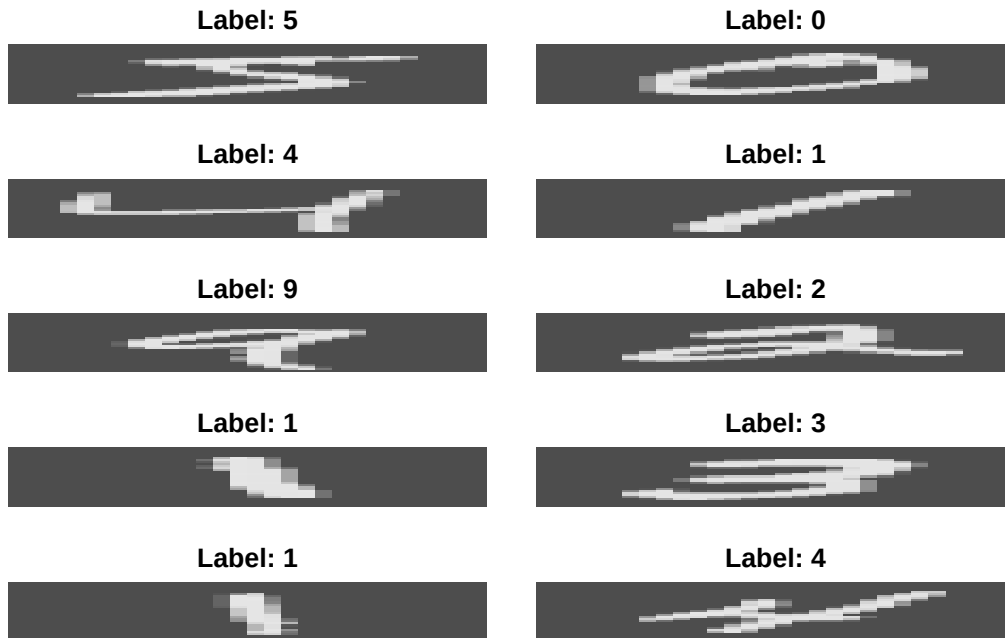
0.B MNIST

MNIST Handwritten Digit Dataset was imported by invoking the function “dataset_mnist()” in Keras library. The training dataset comprises 60,000 grayscale images, while the test dataset comprises 10,000 images. Every image is of size 28×28 pixels. The pixel intensity lies between 0 and 255.

```
[1] 50000    28    28
```

```
[1] 10000    28    28
```

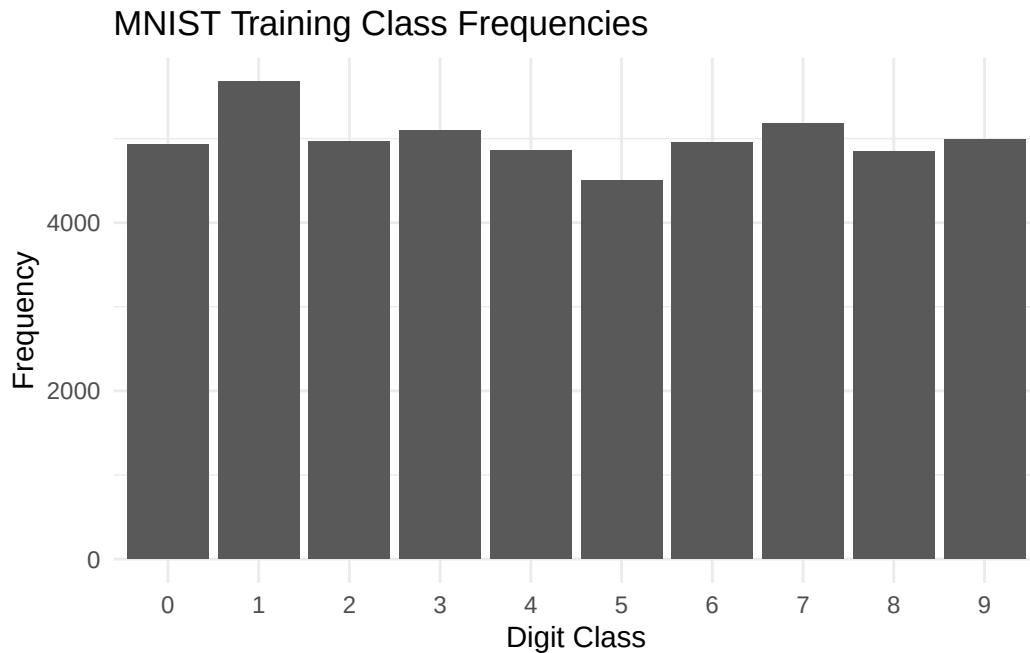
```
[1] 10000    28    28
```



Sample MNIST Images

Five by two grids of sample images for the MNIST dataset were shown alongside their respective label numbers. These images consist of handwritten digits ranging from 0 through 9 presented in grayscale images.

	Digit	Frequency	Percentage
1	0	4932	9.86
2	1	5678	11.36
3	2	4968	9.94
4	3	5101	10.20
5	4	4859	9.72
6	5	4506	9.01
7	6	4951	9.90
8	7	5175	10.35
9	8	4842	9.68
10	9	4988	9.98



```
[1] 50000    784
```

```
[1] 50000     28     28     1
```

```
[1] 50000     10
```

Input Transformations for Neural Networks

For the DNN baseline, each 28×28 image was reshaped into a 784-dimensional vector. This format is suitable for fully connected dense layers, which require one-dimensional input features. For the CNN model, the original spatial image structure was preserved by reshaping each image into a $28 \times 28 \times 1$ tensor. The final dimension represents the single grayscale channel. This format allows convolutional layers to learn local spatial patterns such as edges, curves, and digit strokes. The digit labels were also converted into one-hot encoded vectors with 10 classes, which is required when using categorical cross-entropy loss with a softmax output layer.

Part I: Vanilla Deep Neural Network

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 1680)	0
dense (Dense)	(None, 128)	215,168
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)

Non-trainable params: 0 (0.00 B)

Regression DNN Architecture

A simple deep neural network architecture has been chosen to serve as a base model for forecasting temperatures in Jena. As there are 120 samples of data per hour for each predictive variable, the input was converted from multi-dimensional to one-dimensional and fed into densely connected layers. Two hidden layers using rectified linear unit (ReLU) activation have been utilised in the proposed model. ReLU was chosen as an activation function since it brings nonlinearity into a neural network while mitigating the problem of vanishing gradients in contrast to sigmoid or hyperbolic tangent (tanh). There are 128 neurones in the first hidden layer and 64 neurones in the second hidden layer. One linearly activated neurone is contained in the output layer. Linear activation makes sense for regression, as the dependent variable, 24-hour-ahead temperature, is not bounded and can be either positive or negative. Unlike in classification, probability distribution at the output is not necessary; hence, a linear neurone makes sense.

Design a DNN

DNN Without Regularization

Model: "sequential_1"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

flatten_1 (Flatten)	(None, 1680)	0
dense_3 (Dense)	(None, 128)	215,168
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)

Non-trainable params: 0 (0.00 B)

[1] 223489

Epoch 1/50

200/200 - 4s - 22ms/step - loss: 0.3825 - mae: 0.3825 - val_loss: 0.3839 - val_mae: 0.3839

Epoch 2/50

200/200 - 3s - 16ms/step - loss: 0.2985 - mae: 0.2985 - val_loss: 0.3980 - val_mae: 0.3980

Epoch 3/50

200/200 - 3s - 15ms/step - loss: 0.2732 - mae: 0.2732 - val_loss: 0.3184 - val_mae: 0.3184

Epoch 4/50

200/200 - 3s - 17ms/step - loss: 0.2665 - mae: 0.2665 - val_loss: 0.3932 - val_mae: 0.3932

Epoch 5/50

200/200 - 3s - 14ms/step - loss: 0.2590 - mae: 0.2590 - val_loss: 0.3681 - val_mae: 0.3681

Epoch 6/50

200/200 - 3s - 14ms/step - loss: 0.2474 - mae: 0.2474 - val_loss: 0.3409 - val_mae: 0.3409

Epoch 7/50

200/200 - 3s - 14ms/step - loss: 0.2379 - mae: 0.2379 - val_loss: 0.3119 - val_mae: 0.3119

Epoch 8/50

200/200 - 3s - 17ms/step - loss: 0.2298 - mae: 0.2298 - val_loss: 0.4252 - val_mae: 0.4252

Epoch 9/50

200/200 - 3s - 14ms/step - loss: 0.2255 - mae: 0.2255 - val_loss: 0.3543 - val_mae: 0.3543

Epoch 10/50

200/200 - 3s - 14ms/step - loss: 0.2244 - mae: 0.2244 - val_loss: 0.3638 - val_mae: 0.3638

Epoch 11/50

200/200 - 3s - 16ms/step - loss: 0.2194 - mae: 0.2194 - val_loss: 0.4192 - val_mae: 0.4192

Epoch 12/50

200/200 - 3s - 14ms/step - loss: 0.2137 - mae: 0.2137 - val_loss: 0.3510 - val_mae: 0.3510

DNN With Dropout

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 1680)	0
dense_6 (Dense)	(None, 128)	215,168
dropout (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)

Non-trainable params: 0 (0.00 B)

[1] 223489

Epoch 1/50

200/200 - 3s - 17ms/step - loss: 0.5680 - mae: 0.5680 - val_loss: 0.3993 - val_mae: 0.3993

Epoch 2/50

200/200 - 3s - 14ms/step - loss: 0.3730 - mae: 0.3730 - val_loss: 0.3609 - val_mae: 0.3609

Epoch 3/50

200/200 - 3s - 14ms/step - loss: 0.3418 - mae: 0.3418 - val_loss: 0.3684 - val_mae: 0.3684

Epoch 4/50

200/200 - 3s - 17ms/step - loss: 0.3385 - mae: 0.3385 - val_loss: 0.3468 - val_mae: 0.3468

Epoch 5/50

200/200 - 3s - 14ms/step - loss: 0.3296 - mae: 0.3296 - val_loss: 0.4030 - val_mae: 0.4030

DNN With L2 Weight Decay

Model: "sequential_3"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

flatten_3 (Flatten)	(None, 1680)	0
dense_9 (Dense)	(None, 128)	215,168
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)

Non-trainable params: 0 (0.00 B)

[1] 223489

Epoch 1/50

200/200 - 4s - 19ms/step - loss: 0.7112 - mae: 0.4056 - val_loss: 0.6166 - val_mae: 0.3331

Epoch 2/50

200/200 - 3s - 14ms/step - loss: 0.5608 - mae: 0.2998 - val_loss: 0.6475 - val_mae: 0.4087

Epoch 3/50

200/200 - 3s - 15ms/step - loss: 0.4962 - mae: 0.2780 - val_loss: 0.6421 - val_mae: 0.4439

Epoch 4/50

200/200 - 3s - 14ms/step - loss: 0.4525 - mae: 0.2715 - val_loss: 0.6615 - val_mae: 0.4967

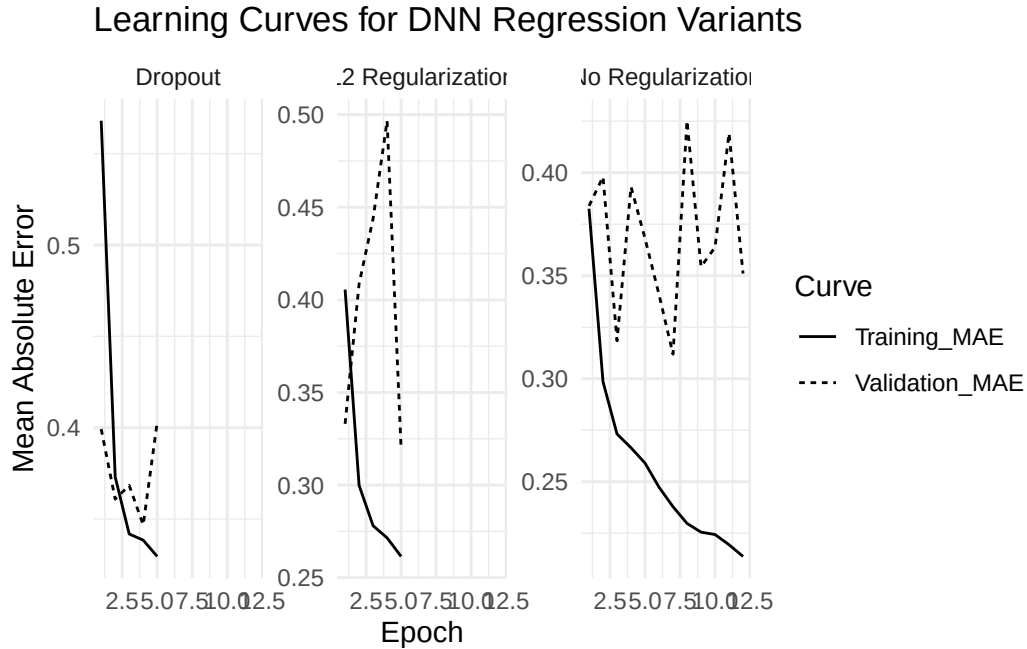
Epoch 5/50

200/200 - 3s - 14ms/step - loss: 0.4122 - mae: 0.2615 - val_loss: 0.4569 - val_mae: 0.3195

Parameter Count Table

	Model	Trainable_Parameters
1	DNN No Regularization	223489
2	DNN Dropout	223489
3	DNN L2 Regularization	223489

Learning Curves for the Three DNN Variants



Training of DNN Regression Models

Training of the three regression models was done via the Adam optimiser and loss function consisting of the mean absolute error. MAE loss was preferred since it represents the mean difference between the predictions and observations, expressed in degrees Celsius. In training the models, early stopping with patience of 5 epochs based on validation MAE was performed. That is, training ended automatically when the model failed to make improvements in validation MAE for five consecutive epochs. Moreover, the best model parameters were restored at the end of the training process to prevent an overfitted model from being retained. In all, three versions of the DNN (Deep Neural Network) were employed. For the first, no regularisation was used, representing the baseline model. The second model included a dropout technique for deactivating a portion of the neurones. The final model used weight decay through an L2 penalty. Learning curves were plotted to identify the bias/variance characteristics of each model. In cases where the no-regularisation model yields lower training MAE but higher validation MAE, then the model is said to be highly variegated due to overfitting. The dropout and L2 techniques are meant to address the issue of overfitting by enhancing regularisation. On the other hand, too much regularisation will result in increased bias and, thus, greater training and validation errors.

Select the Best DNN Variant

No_Regularization	Dropout	L2_Regularization
0.3119193	0.3468305	0.3195390

```
[1] "No_Regularization"
```

Creating Test Arrays For Evaluation

```
[1] 8567 120 14
```

```
[1] 8567
```

PREDICTIONS FROM BEST DNN MODEL

268/268 - 0s - 2ms/step

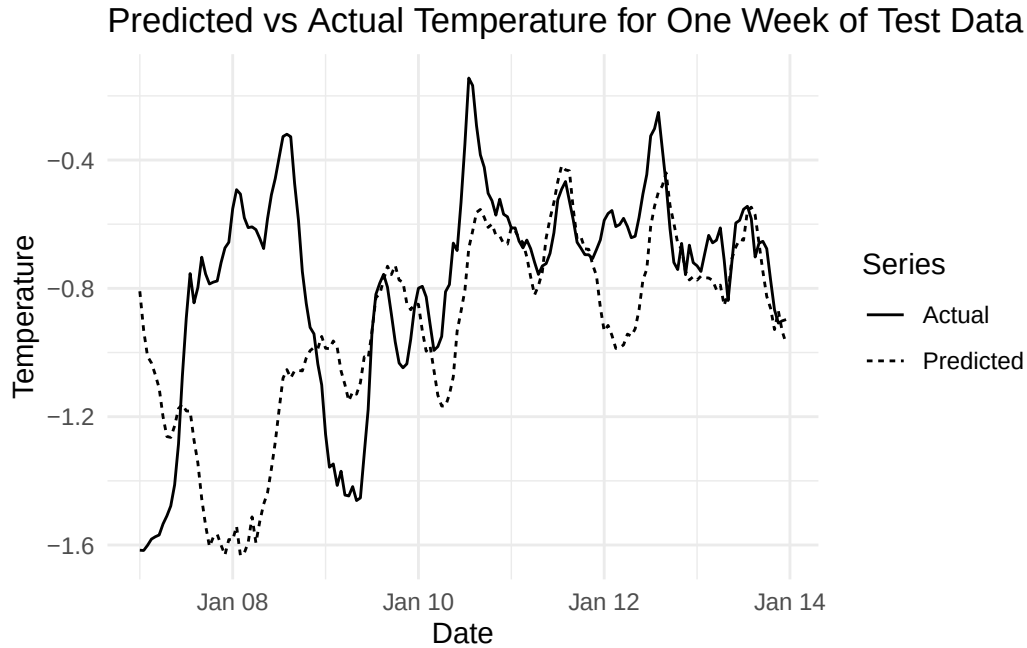
TEST METRICS

	Best_Model	Test_MAE	Test_RMSE	Test_R2
1	No_Regularization	0.3104	0.3975	0.8268

Naive Baseline: Predict $T+24 = T+0$

	Model	Test_MAE
1	Naive Baseline	0.2896
2	Best DNN: No_Regularization	0.3104

One-Week Predicted vs Actual Plot



Test Evaluation of the Best DNN Variant

Selection of the best-performing model relied on the minimum MAE value of the validation samples. As a result, the final model is chosen with regard for its ability to perform well on unseen validation data.

The best DNN performed with an MAE of 0.3104, RMSE of 0.3975 and R^2 of 0.8268 on the test set. The MAE represents the mean of the absolute differences between the predicted values and the target variable in temperature units. RMSE is used to punish models for making predictions that differ considerably from actual results. Finally, R^2 is the share of the 24-hour-ahead temperature variability explained by the model.

A visual interpretation of the predictive power of the model can be provided with the use of a one-week prediction plot. It shows the accuracy of the predictions compared to the changes in the temperature.

Naive baseline predicts 24-hour-ahead temperature by taking into account the temperature at the moment of prediction. Its MAE value equals to 0.2896. A comparison of the MAE values of naive baseline and the best-performing DNN allows concluding about the model's predictive value. In case the MAE of the DNN is lower, it means that a prediction model provides additional insight.

2-D convolutional filter operates on a greyscale image versus 1-D filter operates on a multivariate time-series window

A 2-D convolutional filter operates on a greyscale image by moving across the image in two directions: height and width. At each position, the filter covers a small local patch of pixels, such as a 3×3 region, and computes a weighted sum of the pixel values. This produces a feature map that helps detect spatial patterns such as edges, curves, corners, and digit strokes. A 1-D convolutional filter operates differently because it moves along only one main direction: time. In a multivariate time-series window, the filter slides across consecutive time steps and examines short temporal segments. For example, with hourly Jena climate data, a kernel size of 5 means the filter studies patterns within five consecutive hours across the weather variables. This helps the model detect local temporal patterns such as short-term temperature changes, humidity shifts, and wind-speed variations.

inductive bias and parameter efficiency

Inductive bias shared by both 2D and 1D convolution models is locality in connections as well as parameter sharing. Locality of connections means that features in small neighbourhoods are more correlated than those far apart. In case of an image, the features within close proximity correspond to visual concepts, while, for time series, adjacent points tend to represent a particular pattern. The principle of parameter sharing allows applying the same kernel to different parts of the input data. Rather than training separate weights for all pixels or points, CNN trains a single filter and utilises it multiple times, which results in fewer parameters being trained when compared to traditional DNN.

Why local connectivity suits images and time series but is less natural for tabular data

Local connectivity is well-suited for image data since adjacent pixels belong to the same pattern most of the time. For instance, the strokes used to write a handwritten number are composed of pixels adjacent to each other. Thus, a local filter can extract useful features from them. Local connectivity is also well-suited for time-series data since values that are closer in time tend to be related. For instance, current temperatures are highly related to previous temperatures. Thus, a 1-D filter, which is a type of algorithm that processes data in a single dimension, is able to pick up the relations existing in a short period of time. However, local connectivity is not suited for tabular data because there isn't a spatial or temporal connection between variables in a tabular dataset. For instance, pressure and temperature may be related, yet there isn't a spatial relationship between them in a table. Thus, using local filters in tabular

data may result in missing some relationships between variables that are not adjacent to each other.

II.B(a) 2D-CNN for MNIST Classification

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
conv2d_1 (Conv2D)	(None, 28, 28, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
flatten_4 (Flatten)	(None, 12544)	0
dense_12 (Dense)	(None, 128)	1,605,760
dense_13 (Dense)	(None, 10)	1,290

Total params: 1,625,866 (6.20 MB)

Trainable params: 1,625,866 (6.20 MB)

Non-trainable params: 0 (0.00 B)

Interpretation

The 2D-CNN architecture utilises the input shape (28, 28, 1) where the images are 28×28 in size and represent greyscale images with only one colour channel. The model has two Conv2D layers followed by rectified linear unit (ReLU) activation and same-padding. This allows for extracting spatially localised features such as edges, curves, and strokes from the digits. The MaxPooling2D layer allows for dimensionality reduction of the output but preserves important features from the previous layers. Lastly, the fully connected layer is used to classify the input data into one of the 10 classes using softmax activation.

II.B(b) COMPILE CNN MODEL

TRAINING TIME PER EPOCH

```
Epoch 1/20
391/391 - 37s - 94ms/step - accuracy: 0.9449 - loss: 0.1839 - val_accuracy: 0.9844 - val_loss: 0.0383
Epoch 2/20
391/391 - 37s - 94ms/step - accuracy: 0.9853 - loss: 0.0484 - val_accuracy: 0.9860 - val_loss: 0.0383
Epoch 3/20
391/391 - 37s - 94ms/step - accuracy: 0.9913 - loss: 0.0293 - val_accuracy: 0.9867 - val_loss: 0.0383
Epoch 4/20
391/391 - 36s - 92ms/step - accuracy: 0.9930 - loss: 0.0205 - val_accuracy: 0.9879 - val_loss: 0.0383
Epoch 5/20
391/391 - 36s - 92ms/step - accuracy: 0.9958 - loss: 0.0132 - val_accuracy: 0.9888 - val_loss: 0.0383
Epoch 6/20
391/391 - 37s - 94ms/step - accuracy: 0.9964 - loss: 0.0103 - val_accuracy: 0.9873 - val_loss: 0.0383
Epoch 7/20
391/391 - 35s - 90ms/step - accuracy: 0.9969 - loss: 0.0092 - val_accuracy: 0.9873 - val_loss: 0.0383
Epoch 8/20
391/391 - 36s - 93ms/step - accuracy: 0.9978 - loss: 0.0061 - val_accuracy: 0.9880 - val_loss: 0.0383
Epoch 9/20
391/391 - 36s - 92ms/step - accuracy: 0.9979 - loss: 0.0062 - val_accuracy: 0.9876 - val_loss: 0.0383
```

```
[1] 36.29407
```

TEST ACCURACY

```
313/313 - 2s - 7ms/step - accuracy: 0.9884 - loss: 0.0383
```

```
$accuracy
```

```
[1] 0.9884
```

PREDICTIONS AND CONFUSION MATRIX

```
313/313 - 2s - 7ms/step
```

Actual \ Predicted										
	0	1	2	3	4	5	6	7	8	9
0	970	0	2	1	0	2	3	0	2	0
1	0	1134	1	0	0	0	0	0	0	0
2	0	5	1021	0	0	0	0	4	2	0
3	0	1	1	1002	0	5	0	0	1	0
4	0	1	0	0	970	0	1	0	1	9
5	0	0	1	5	0	885	1	0	0	0
6	5	2	0	1	1	9	936	0	4	0
7	0	3	8	2	0	1	0	1006	3	5
8	1	0	4	0	2	2	0	1	962	2
9	1	2	0	2	1	4	0	0	1	998

MACRO-F1 FUNCTION

	Model	Dataset	Test_Accuracy	Macro_F1	Train_Time_Per_EPOCH_Seconds
1	2D-CNN	MNIST	0.9884	0.9883	36.29

Interpretation

The 2D-CNN was trained using the Adam optimizer and categorical cross-entropy loss, consistent with the DNN classification model in Part I. Early stopping with patience 5 was used to stop training when validation loss stopped improving. The model achieved a test accuracy of 0.9884 and a macro-F1 score of 0.9883. The confusion matrix shows the distribution of correct and incorrect predictions across the ten digit classes.

PARAMETER COUNT FUNCTION

[1] 1625866

II.C(a) 1D-CNN FOR JENA REGRESSION

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 120, 32)	2,272

conv1d_1 (Conv1D)	(None, 120, 64)	10,304
max_pooling1d (MaxPooling1D)	(None, 60, 64)	0
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense_14 (Dense)	(None, 64)	4,160
dense_15 (Dense)	(None, 1)	65

Total params: 16,801 (65.63 KB)
Trainable params: 16,801 (65.63 KB)
Non-trainable params: 0 (0.00 B)

Interpretaton

The architecture of the 1D-CNN was intended for inputs of size 120 x N, where the integer 120 refers to the last 120 hourly records, and the other dimension corresponds to the predictor variables of the atmospheric features. There are two layers of Conv1D layers activated by the ReLU function, followed by one MaxPooling1D layer, one GlobalAveragePooling1D layer, and finally a single output neurone using a linear activation function since we are performing regression.

b.Justify kernel size and temporal receptive field

The Conv1D filters have a kernel size of 5. As Jena's dataset has been resampled to hourly data, this implies that each of the convolution kernels considers five hours' local temporal context. This ensures that the model can identify local weather patterns such as rising temperatures, cooling temperatures, humidity, and atmospheric pressure. Adding a second layer of Conv1D filters enables the model to consider a larger temporal window. The first layer extracts local five-hour windows, while the second layer fuses those patterns to identify more general temporal patterns. Consequently, while each convolution filter considers only five hours of temporal data, by stacking them together, the model is able to identify more complex patterns over a larger window of 120 hours. A kernel size of 5 was selected since this value is adequate to capture short-term weather variability without oversmoothing the time series data.

II.C(c) COMPILE 1D-CNN

TRAIN 1D-CNN

```
Epoch 1/50
200/200 - 5s - 25ms/step - loss: 0.4521 - mae: 0.4521 - val_loss: 0.3828 - val_mae: 0.3828
Epoch 2/50
200/200 - 5s - 23ms/step - loss: 0.4154 - mae: 0.4154 - val_loss: 0.4061 - val_mae: 0.4061
Epoch 3/50
200/200 - 5s - 26ms/step - loss: 0.3709 - mae: 0.3709 - val_loss: 0.3676 - val_mae: 0.3676
Epoch 4/50
200/200 - 4s - 21ms/step - loss: 0.3066 - mae: 0.3066 - val_loss: 0.3837 - val_mae: 0.3837
Epoch 5/50
200/200 - 4s - 21ms/step - loss: 0.2838 - mae: 0.2838 - val_loss: 0.4417 - val_mae: 0.4417
Epoch 6/50
200/200 - 5s - 23ms/step - loss: 0.2796 - mae: 0.2796 - val_loss: 0.3489 - val_mae: 0.3489
Epoch 7/50
200/200 - 4s - 21ms/step - loss: 0.2690 - mae: 0.2690 - val_loss: 0.3986 - val_mae: 0.3986
Epoch 8/50
200/200 - 4s - 20ms/step - loss: 0.2684 - mae: 0.2684 - val_loss: 0.4089 - val_mae: 0.4089
Epoch 9/50
200/200 - 5s - 23ms/step - loss: 0.2633 - mae: 0.2633 - val_loss: 0.4065 - val_mae: 0.4065
Epoch 10/50
200/200 - 4s - 21ms/step - loss: 0.2600 - mae: 0.2600 - val_loss: 0.3777 - val_mae: 0.3777
Epoch 11/50
200/200 - 4s - 21ms/step - loss: 0.2609 - mae: 0.2609 - val_loss: 0.3649 - val_mae: 0.3649
```

TEST EVALUATION

```
268/268 - 1s - 2ms/step
```

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters	Train_Time_Per_Epoch_Seconds
1	1D-CNN	0.2943	0.3773	0.8439	16801	4.45

COMPARE 1D-CNN WITH DNN BASELINE

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters
1	DNN Baseline	0.3104	0.3975	0.8268	223489
2	1D-CNN	0.2943	0.3773	0.8439	16801

Train_Time_Per_Epoch_Seconds	
1	NA
2	4.45

Interpretation

The 1D-CNN was trained using Adam optimization and MAE loss, following the same evaluation setting as the DNN regression baseline. On the Jena test set, the model achieved a MAE of 0.2943, RMSE of 0.3773, and R^2 of 0.8439. Compared with the DNN baseline, the 1D-CNN preserves the temporal structure of the 120-hour input window. The DNN flattens the time-series input and treats all lagged values as ordinary features, while the 1D-CNN applies filters across consecutive hours. Therefore, the 1D-CNN is theoretically better suited to capturing local temporal dependencies in climate data. If the 1D-CNN achieves a lower MAE and RMSE or a higher R^2 than the DNN, this indicates that exploiting temporal locality improves forecasting performance.

Part III: Recurrent Neural Network (LSTM)

III.A Theoretical Motivation

a. Why a vanilla SimpleRNN struggles with sequences of 120+ steps

The standard vanilla Simple Recurrent Neural Network has problems processing sequences that are too long, since it requires the passing of information through a recurrent function repeatedly at each time step. When backpropagation through time occurs, the gradient gets multiplied repeatedly by recurrent weight matrices. In case these multiplications produce values less than one, then the gradient gets vanishingly small during backpropagation.

b. How LSTM gates carry information over long horizons

The Long Short-Term Memory network solves the problem of vanishing gradients using the concept of cell state and three types of gates: the forget gate, the input gate, and the output gate. The function of the forget gate is to identify whether information stored previously in the cell state must be forgotten or kept. The input gate determines the amount of information to add to the cell state. The output gate regulates the amount of information that is transferred from the cell state to the hidden state for use in predictions. Cell state enables the movement of information through several time steps with fewer non-linear transformations. For this reason, it becomes easier for LSTM to remember critical information in the long term. The concept is beneficial in the Jena climate example because future temperature at 24 hours could depend

not only on the immediate one hour but also on daily and multi-day patterns in the lookback window of 120 hours.

c. Why treating MNIST rows as time steps is valid but suboptimal

Treating MNIST images as sequences is valid because each image can be represented as 28 rows, where each row contains 28 pixel values. Under this representation, the LSTM processes the image row by row as a sequence of 28 time steps with 28 features at each step. However, this is a suboptimal use of LSTM because MNIST is primarily a spatial image problem, not a temporal sequence problem. The arrangement of pixels contains two-dimensional spatial structure, including local strokes, curves, and edges. A CNN is better suited to this structure because its filters move across both height and width and directly capture local spatial patterns. Using an LSTM for MNIST shows the importance of inductive bias. Even if an LSTM has enough capacity to classify digits, its sequential bias does not match the image structure as naturally as the spatial locality and parameter sharing bias of a CNN. Therefore, the CNN is expected to outperform the LSTM on MNIST classification.

III.B(a) LSTM MODEL FOR JENA REGRESSION

Model: "sequential_6"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 120, 64)	20,224
lstm_1 (LSTM)	(None, 32)	12,416
dense_16 (Dense)	(None, 1)	33

Total params: 32,673 (127.63 KB)

Trainable params: 32,673 (127.63 KB)

Non-trainable params: 0 (0.00 B)

Interpretation

Input for the LSTM model was set to be of size (120, number of predictors) since the lookback period is of 120 hours. Two LSTM layers were added, and the first layer's return sequences attribute was set to True in order for the second LSTM layer to process the whole temporal information. Recurrent dropout was incorporated into the model to avoid overfitting

by randomly dropping units from the recurrent neural network. Linear activation function was chosen for the output layer due to the regression problem.

b. Train and Evaluate LSTM(COMPILE LSTM MODEL)

TRAIN LSTM

Epoch 1/50

200/200 - 27s - 135ms/step - loss: 0.3540 - mae: 0.3540 - val_loss: 0.4035 - val_mae: 0.4035

Epoch 2/50

200/200 - 25s - 126ms/step - loss: 0.3033 - mae: 0.3033 - val_loss: 0.4226 - val_mae: 0.4226

Epoch 3/50

200/200 - 19s - 97ms/step - loss: 0.2943 - mae: 0.2943 - val_loss: 0.3543 - val_mae: 0.3543

Epoch 4/50

200/200 - 21s - 103ms/step - loss: 0.2868 - mae: 0.2868 - val_loss: 0.3648 - val_mae: 0.3648

Epoch 5/50

200/200 - 19s - 97ms/step - loss: 0.2827 - mae: 0.2827 - val_loss: 0.3995 - val_mae: 0.3995

Epoch 6/50

200/200 - 19s - 96ms/step - loss: 0.2780 - mae: 0.2780 - val_loss: 0.3511 - val_mae: 0.3511

Epoch 7/50

200/200 - 20s - 99ms/step - loss: 0.2755 - mae: 0.2755 - val_loss: 0.3445 - val_mae: 0.3445

Epoch 8/50

200/200 - 20s - 98ms/step - loss: 0.2717 - mae: 0.2717 - val_loss: 0.3447 - val_mae: 0.3447

Epoch 9/50

200/200 - 22s - 111ms/step - loss: 0.2699 - mae: 0.2699 - val_loss: 0.3397 - val_mae: 0.3397

Epoch 10/50

200/200 - 21s - 103ms/step - loss: 0.2653 - mae: 0.2653 - val_loss: 0.3554 - val_mae: 0.3554

Epoch 11/50

200/200 - 20s - 99ms/step - loss: 0.2652 - mae: 0.2652 - val_loss: 0.3448 - val_mae: 0.3448

Epoch 12/50

200/200 - 20s - 100ms/step - loss: 0.2627 - mae: 0.2627 - val_loss: 0.3561 - val_mae: 0.3561

Epoch 13/50

200/200 - 21s - 105ms/step - loss: 0.2608 - mae: 0.2608 - val_loss: 0.3488 - val_mae: 0.3488

Epoch 14/50

200/200 - 19s - 96ms/step - loss: 0.2592 - mae: 0.2592 - val_loss: 0.3419 - val_mae: 0.3419

TEST EVALUATION

268/268 - 4s - 16ms/step

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters	Train_Time_Per_Epoch_Seconds
1	LSTM	0.271	0.3474	0.8677	32673	20.93

c. Ablation: SimpleRNN vs LSTM

TRAIN SIMPLE RNN

```
Epoch 1/50
200/200 - 9s - 44ms/step - loss: 0.3262 - mae: 0.3262 - val_loss: 0.4273 - val_mae: 0.4273
Epoch 2/50
200/200 - 8s - 40ms/step - loss: 0.3032 - mae: 0.3032 - val_loss: 0.4002 - val_mae: 0.4002
Epoch 3/50
200/200 - 8s - 40ms/step - loss: 0.2952 - mae: 0.2952 - val_loss: 0.4193 - val_mae: 0.4193
Epoch 4/50
200/200 - 9s - 43ms/step - loss: 0.2936 - mae: 0.2936 - val_loss: 0.4097 - val_mae: 0.4097
Epoch 5/50
200/200 - 9s - 43ms/step - loss: 0.2892 - mae: 0.2892 - val_loss: 0.4284 - val_mae: 0.4284
```

TEST PERFORMANCE OF SIMPLE RNN

```
268/268 - 2s - 7ms/step
```

```
[1] 0.3005973
```

Interpretation LSTM vs SimpleRNN for Jena Regression

The training process for the LSTM model was performed in the same way as for the DNN model. The test results show that the LSTM network reached the values of MAE, RMSE, and R^2 equal to 0.271, and 0.3474, respectively. In order to find out how important the property of memory is for the LSTM, an ablation model based on the SimpleRNN with the same structure as the LSTM model was trained. The result obtained from the SimpleRNN showed that the test MAE equals 0.3006. Thus, the reason for the better performance of the LSTM model is associated with overcoming the problem of the vanishing gradient that occurs while training the RNN. Indeed, during training, when passing through many steps, the gradient loses its value and makes it difficult for the RNN model to learn some patterns in the sequence. This is overcome by the LSTM model using its cell state.

III.C LSTM for Jena Classification

a.Create classification generator and LSTM model

CLASSIFICATION DATA GENERATOR

CREATE CLASSIFICATION GENERATORS

```
[1] 128 120 14
```

```
[1] 128 4
```

LSTM MODEL FOR JENA CLASSIFICATION

```
Model: "sequential_8"
```

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 120, 64)	20,224
lstm_3 (LSTM)	(None, 32)	12,416
dense_18 (Dense)	(None, 4)	132

```
Total params: 32,772 (128.02 KB)
```

```
Trainable params: 32,772 (128.02 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

Interpretation

As for the Jena classification problem, a softmax layer with four neurones was used instead of the regression output layer. The first, second, third, and fourth neurones represent the categories cold, cool, mild, and warm temperatures, respectively. The softmax layer is suitable since its purpose is to transform the output of the model into probabilities summing up to one. The loss function was also switched from MAE to categorical cross-entropy.

b. Train and evaluate: accuracy, macro-F1, confusion matrix

TRAIN LSTM CLASSIFICATION MODEL

```
Epoch 1/50
200/200 - 22s - 111ms/step - accuracy: 0.5589 - loss: 0.9960 - val_accuracy: 0.8594 - val_loss: 0.8594
Epoch 2/50
200/200 - 19s - 96ms/step - accuracy: 0.6124 - loss: 0.8536 - val_accuracy: 0.2266 - val_loss: 0.2266
Epoch 3/50
200/200 - 20s - 100ms/step - accuracy: 0.6379 - loss: 0.8249 - val_accuracy: 0.2344 - val_loss: 0.2344
Epoch 4/50
200/200 - 21s - 103ms/step - accuracy: 0.6122 - loss: 0.8818 - val_accuracy: 0.8594 - val_loss: 0.8594
Epoch 5/50
200/200 - 20s - 99ms/step - accuracy: 0.6409 - loss: 0.8132 - val_accuracy: 0.3516 - val_loss: 0.3516
Epoch 6/50
200/200 - 19s - 97ms/step - accuracy: 0.6623 - loss: 0.7637 - val_accuracy: 0.5547 - val_loss: 0.5547
Epoch 7/50
200/200 - 19s - 95ms/step - accuracy: 0.5807 - loss: 0.9854 - val_accuracy: 0.8594 - val_loss: 0.8594
Epoch 8/50
200/200 - 20s - 99ms/step - accuracy: 0.6186 - loss: 0.8586 - val_accuracy: 0.4766 - val_loss: 0.4766
Epoch 9/50
200/200 - 19s - 97ms/step - accuracy: 0.6339 - loss: 0.8333 - val_accuracy: 0.8594 - val_loss: 0.8594
Epoch 10/50
200/200 - 19s - 94ms/step - accuracy: 0.6291 - loss: 0.8316 - val_accuracy: 0.4766 - val_loss: 0.4766
Epoch 11/50
200/200 - 26s - 130ms/step - accuracy: 0.6388 - loss: 0.8093 - val_accuracy: 0.4766 - val_loss: 0.4766
Epoch 12/50
200/200 - 22s - 110ms/step - accuracy: 0.6323 - loss: 0.8082 - val_accuracy: 0.4766 - val_loss: 0.4766
```

CREATE TEST CLASSIFICATION ARRAYS

TEST PREDICTION

```
267/267 - 4s - 16ms/step
```

ACCURACY, MACRO-F1, CONFUSION MATRIX

Model	Dataset	Test_Accuracy	Macro_F1
-------	---------	---------------	----------

1 LSTM Jena 4-Class 0.6557 0.3666

Predicted				
Actual	Cold	Cool	Mild	Warm
Cold	0	767	0	0
Cool	0	3194	476	0
Mild	0	584	2408	0
Warm	0	22	1093	0

Interpretation

The accuracy for the LSTM classification model was $\text{round}(\text{lstm_class_accuracy}, 4)$ while the macro F1-score was $\text{round}(\text{lstm_class_macro_f1}, 4)$. The confusion matrix displays the classification of the four temperature classes. It is likely that misclassifications will happen between classes that have nearby temperature ranges, such as Cool vs. Mild and Mild vs. Warm.

c. Discussion: Regression or classification advantage?

The LSTM model would show a greater advantage over the DNN in the case of the regression problem than over the classification problem. When dealing with a regression problem, the LSTM model predicts the temperature for the next 24 hours, hence making use of learned temporal relationships, daily cycles, and weather patterns over a period of 120 hours. On the other hand, the regression output is changed into four broad classes when using the classification model. As a result, valuable information is lost since temperatures near the boundary of each category could be classified into different classes despite having similar numerical values. An example is that 9.9°C would be classified as cool, whereas 10.0°C would fall into the mild class despite there being no significant difference between the temperatures. Ideally, LSTM models are meant to capture temporal information in their memory gates. In this respect, LSTM models would have an edge over the DNN on account of the temporal aspect.

III.D LSTM for MNIST Sequential Image

a. Reshape MNIST images to (28, 28)

```
[1] 50000    28    28
```



```
[1] 10000      28      28
```

```
[1] 10000      28      28
```

Interpretation

The images from the MNIST dataset were converted to have a series of 28 timesteps, with each timestep consisting of one row of pixels of size 28 x 28. Therefore, each image used for training had an input shape of 28 timesteps, with each timestep representing one row of pixels.

b. Design and train LSTM model

```
Model: "sequential_9"
```

Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 64)	23,808
dense_19 (Dense)	(None, 128)	8,320
dense_20 (Dense)	(None, 10)	1,290

```
Total params: 33,418 (130.54 KB)
```

```
Trainable params: 33,418 (130.54 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

COMPILE MNIST LSTM

TRAIN MNIST LSTM

```
Epoch 1/20
```

```
391/391 - 9s - 23ms/step - accuracy: 0.6973 - loss: 0.8872 - val_accuracy: 0.9127 - val_loss
```

```
Epoch 2/20
```

```
391/391 - 5s - 13ms/step - accuracy: 0.8936 - loss: 0.3278 - val_accuracy: 0.9478 - val_loss
```

```
Epoch 3/20
```

```
391/391 - 5s - 13ms/step - accuracy: 0.9283 - loss: 0.2251 - val_accuracy: 0.9593 - val_loss
```

```
Epoch 4/20
```

```
391/391 - 5s - 13ms/step - accuracy: 0.9452 - loss: 0.1766 - val_accuracy: 0.9658 - val_loss
```

```

Epoch 5/20
391/391 - 6s - 14ms/step - accuracy: 0.9543 - loss: 0.1466 - val_accuracy: 0.9729 - val_loss
Epoch 6/20
391/391 - 5s - 14ms/step - accuracy: 0.9608 - loss: 0.1267 - val_accuracy: 0.9774 - val_loss
Epoch 7/20
391/391 - 5s - 13ms/step - accuracy: 0.9650 - loss: 0.1128 - val_accuracy: 0.9790 - val_loss
Epoch 8/20
391/391 - 5s - 13ms/step - accuracy: 0.9682 - loss: 0.1013 - val_accuracy: 0.9808 - val_loss
Epoch 9/20
391/391 - 5s - 12ms/step - accuracy: 0.9709 - loss: 0.0920 - val_accuracy: 0.9825 - val_loss
Epoch 10/20
391/391 - 5s - 12ms/step - accuracy: 0.9726 - loss: 0.0868 - val_accuracy: 0.9828 - val_loss
Epoch 11/20
391/391 - 5s - 13ms/step - accuracy: 0.9758 - loss: 0.0773 - val_accuracy: 0.9844 - val_loss
Epoch 12/20
391/391 - 5s - 12ms/step - accuracy: 0.9763 - loss: 0.0743 - val_accuracy: 0.9831 - val_loss
Epoch 13/20
391/391 - 5s - 13ms/step - accuracy: 0.9787 - loss: 0.0692 - val_accuracy: 0.9845 - val_loss
Epoch 14/20
391/391 - 5s - 12ms/step - accuracy: 0.9790 - loss: 0.0666 - val_accuracy: 0.9855 - val_loss
Epoch 15/20
391/391 - 5s - 12ms/step - accuracy: 0.9803 - loss: 0.0601 - val_accuracy: 0.9849 - val_loss
Epoch 16/20
391/391 - 5s - 12ms/step - accuracy: 0.9810 - loss: 0.0605 - val_accuracy: 0.9864 - val_loss
Epoch 17/20
391/391 - 5s - 12ms/step - accuracy: 0.9818 - loss: 0.0556 - val_accuracy: 0.9862 - val_loss
Epoch 18/20
391/391 - 5s - 13ms/step - accuracy: 0.9831 - loss: 0.0529 - val_accuracy: 0.9848 - val_loss
Epoch 19/20
391/391 - 5s - 12ms/step - accuracy: 0.9836 - loss: 0.0510 - val_accuracy: 0.9869 - val_loss
Epoch 20/20
391/391 - 5s - 12ms/step - accuracy: 0.9839 - loss: 0.0496 - val_accuracy: 0.9876 - val_loss

```

c. Test accuracy and comparison with CNN

```

313/313 - 1s - 3ms/step - accuracy: 0.9882 - loss: 0.0385

```

```

$accuracy
[1] 0.9882

```

PARAMETER COUNT

[1] 33418

COMPARE LSTM WITH CNN

	Model	Test_Accuracy	Parameters	Train_Time_Per_Epoch_Seconds
1	2D-CNN	0.9884	1625866	36.29
2	LSTM Sequential Image	0.9882	33418	5.21

Interpretation

The accuracy of the MNIST LSTM model was 0.98. In this case, the images were considered sequences of 28 rows, with each row being comprised of 28 pixel features. It should be noted that the LSTM is likely to do poorly compared to the 2D-CNN on MNIST since handwritten digit classification is primarily based on learning spatial features as opposed to a temporal sequence. This is because a CNN learns spatial patterns such as edges, strokes, and curves by filtering over the height and width dimensions. On the other hand, the LSTM learns the image one row at a time. It can be seen from the above comparison that inductive bias plays an essential role in deep learning models. While the LSTM probably has sufficient model complexity to solve the problem of digit classification, it suffers from poor inductive bias due to the nature of the task.

Part IV.1 Design Brief — Custom Hybrid Architecture

For this particular task, the chosen architecture for the custom hybrid network is a CNN-LSTM model for predicting temperatures in Jena. The proposed architecture consists of two components – a 1D-CNN and an LSTM network. The rationale behind this particular design is the weakness of the separate models. Firstly, the baseline DNN assumes that the 120-hour window will be considered as one huge vector. In other words, the order of the elements will be ignored, and the algorithm will not be able to learn any local time-related patterns. The proposed architecture tries to compensate for this by using convolutional layers, which help detect short-term local patterns, including hourly changes in temperature, humidity and pressure. Although CNN is a good model, there may still be issues in learning longer-range dependencies during five days of input. The proposed model should be more powerful in detecting long-term dependencies due to the use of the LSTM part. In addressing these deficiencies, the hybrid between CNN and LSTM leverages the benefits offered by the two neural networks. Conv1D layers help identify local temporal patterns within the multivariate

climate sequence. Then, the identified features pass through the LSTM layer, where they help in building long-term dependencies in the transformed sequence. In other words, the hybrid model adopts two inductive biases, one for local temporal feature extraction using convolution and the other for long-term sequential dependencies using LSTM gates. Using the regression of the Jena temperature is ideal for evaluating the performance of such a hybrid since the weather data contains short- and long-term temporal patterns. Short-term patterns can emerge in a matter of a few hours, while daily patterns or even weather patterns over multiple days can span the 120-hour period considered. Given that the goal is predicting the temperature for 24 hours ahead, such a model should be effective.

IV.1 Design Brief (Using Suggested Hybrid Pattern: CNN–LSTM)

IV.1 Design Brief

The CNN–LSTM hybrid approach used in this project is based on the combination of 1D convolutional and LSTM layers for the regression of the temperature dataset from the city of Jena. This hybrid design was selected because of the disadvantages that come with using a single architecture such as DNN, CNN, or LSTM. The DNN approach does not consider the time dimension of the dataset since it flattens the data before training. The 1D-CNN approach outperforms the baseline model by modeling the local temporal dependency of the data; however, it fails to model the long-term temporal dependencies of the data over the 120 hours of the look-back window. The LSTM network can learn the long-term dependencies using the gating approach, but it may not perform optimally in extracting short-term dependencies like the convolutional approach. This CNN-LSTM combination makes up for the drawbacks of both architectures. First of all, the Conv1D layers will be responsible for extracting local temporal characteristics like, for example, the variations in weather within a certain time, that is temperature or humidity variations from hour to hour. Then the feature sequence will be fed into the LSTM layer, which will help identify any temporal relationships and long-range dependencies. Thus, two inductive biases are created in one architecture, namely locality (CNN) and long-term memory (LSTM). The Jena climate dataset is suitable for testing the performance of this combination of CNN and LSTM because there exist both short-term weather changes and long-term ones as well. Moreover, the purpose of this task is to forecast the temperature after 24 hours based on the information for the previous 120 hours so that the hybrid model will be able to use both locality and sequentiality.

Part IV: Custom Hybrid Architecture

Model: "CNN_LSTM_Hybrid_Jena_Regression"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

Input_120_hours (InputLayer)	(None, 120, 14)	0
Conv1D_Local_Features_1 (Conv1D)	(None, 120, 32)	2,272
Conv1D_Local_Features_2 (Conv1D)	(None, 120, 64)	10,304
MaxPooling_Temporal_Reduction (MaxPooling1D)	(None, 60, 64)	0
LSTM_Long_Range_Dependencies (LSTM)	(None, 64)	33,024
Dense_Feature_Integration (Dense)	(None, 64)	4,160
Temperature_Output (Dense)	(None, 1)	65
Total params: 49,825 (194.63 KB)		
Trainable params: 49,825 (194.63 KB)		
Non-trainable params: 0 (0.00 B)		

COMPILE HYBRID MODEL

TRAIN HYBRID MODEL

Epoch 1/50
200/200 - 13s - 63ms/step - loss: 0.3417 - mae: 0.3417 - val_loss: 0.4418 - val_mae: 0.4418
Epoch 2/50
200/200 - 10s - 52ms/step - loss: 0.2960 - mae: 0.2960 - val_loss: 0.4574 - val_mae: 0.4574
Epoch 3/50
200/200 - 12s - 58ms/step - loss: 0.2827 - mae: 0.2827 - val_loss: 0.4051 - val_mae: 0.4051
Epoch 4/50
200/200 - 10s - 49ms/step - loss: 0.2795 - mae: 0.2795 - val_loss: 0.3477 - val_mae: 0.3477
Epoch 5/50
200/200 - 10s - 51ms/step - loss: 0.2759 - mae: 0.2759 - val_loss: 0.3601 - val_mae: 0.3601
Epoch 6/50
200/200 - 10s - 50ms/step - loss: 0.2704 - mae: 0.2704 - val_loss: 0.3856 - val_mae: 0.3856
Epoch 7/50
200/200 - 9s - 47ms/step - loss: 0.2676 - mae: 0.2676 - val_loss: 0.3649 - val_mae: 0.3649
Epoch 8/50
200/200 - 10s - 51ms/step - loss: 0.2634 - mae: 0.2634 - val_loss: 0.3727 - val_mae: 0.3727

Epoch 9/50
200/200 - 9s - 47ms/step - loss: 0.2613 - mae: 0.2613 - val_loss: 0.3397 - val_mae: 0.3397
Epoch 10/50
200/200 - 9s - 47ms/step - loss: 0.2582 - mae: 0.2582 - val_loss: 0.3550 - val_mae: 0.3550
Epoch 11/50
200/200 - 10s - 50ms/step - loss: 0.2514 - mae: 0.2514 - val_loss: 0.3227 - val_mae: 0.3227
Epoch 12/50
200/200 - 10s - 50ms/step - loss: 0.2489 - mae: 0.2489 - val_loss: 0.3364 - val_mae: 0.3364
Epoch 13/50
200/200 - 10s - 48ms/step - loss: 0.2459 - mae: 0.2459 - val_loss: 0.3222 - val_mae: 0.3222
Epoch 14/50
200/200 - 10s - 49ms/step - loss: 0.2428 - mae: 0.2428 - val_loss: 0.3195 - val_mae: 0.3195
Epoch 15/50
200/200 - 10s - 48ms/step - loss: 0.2365 - mae: 0.2365 - val_loss: 0.3116 - val_mae: 0.3116
Epoch 16/50
200/200 - 10s - 52ms/step - loss: 0.2308 - mae: 0.2308 - val_loss: 0.3148 - val_mae: 0.3148
Epoch 17/50
200/200 - 10s - 49ms/step - loss: 0.2283 - mae: 0.2283 - val_loss: 0.3058 - val_mae: 0.3058
Epoch 18/50
200/200 - 10s - 48ms/step - loss: 0.2235 - mae: 0.2235 - val_loss: 0.3245 - val_mae: 0.3245
Epoch 19/50
200/200 - 9s - 47ms/step - loss: 0.2170 - mae: 0.2170 - val_loss: 0.3156 - val_mae: 0.3156
Epoch 20/50
200/200 - 9s - 47ms/step - loss: 0.2157 - mae: 0.2157 - val_loss: 0.2998 - val_mae: 0.2998
Epoch 21/50
200/200 - 10s - 51ms/step - loss: 0.2098 - mae: 0.2098 - val_loss: 0.3087 - val_mae: 0.3087
Epoch 22/50
200/200 - 10s - 49ms/step - loss: 0.2026 - mae: 0.2026 - val_loss: 0.2896 - val_mae: 0.2896
Epoch 23/50
200/200 - 9s - 47ms/step - loss: 0.2001 - mae: 0.2001 - val_loss: 0.3088 - val_mae: 0.3088
Epoch 24/50
200/200 - 10s - 48ms/step - loss: 0.1969 - mae: 0.1969 - val_loss: 0.3252 - val_mae: 0.3252
Epoch 25/50
200/200 - 9s - 47ms/step - loss: 0.1920 - mae: 0.1920 - val_loss: 0.3047 - val_mae: 0.3047
Epoch 26/50
200/200 - 10s - 49ms/step - loss: 0.1904 - mae: 0.1904 - val_loss: 0.3364 - val_mae: 0.3364
Epoch 27/50
200/200 - 10s - 49ms/step - loss: 0.1842 - mae: 0.1842 - val_loss: 0.3775 - val_mae: 0.3775

CNN-LSTM Hybrid Architecture Diagram

Input Layer

(120, number of predictors)

Conv1D Layer 1

32 filters, kernel size = 5, ReLU, same padding

Conv1D Layer 2

64 filters, kernel size = 5, ReLU, same padding

MaxPooling1D

pool size = 2

LSTM Layer

64 units, recurrent dropout = 0.20

Dense Layer

64 neurons, ReLU

Output Layer

1 neuron, linear activation

Implementation of the CNN-LSTM Hybrid Architecture

The CNN-LSTM hybrid model was built using the Keras Functional API, satisfying the project requirements to develop a non-sequential model architecture, which should be constructed explicitly using the ‘function’ and connected layers.

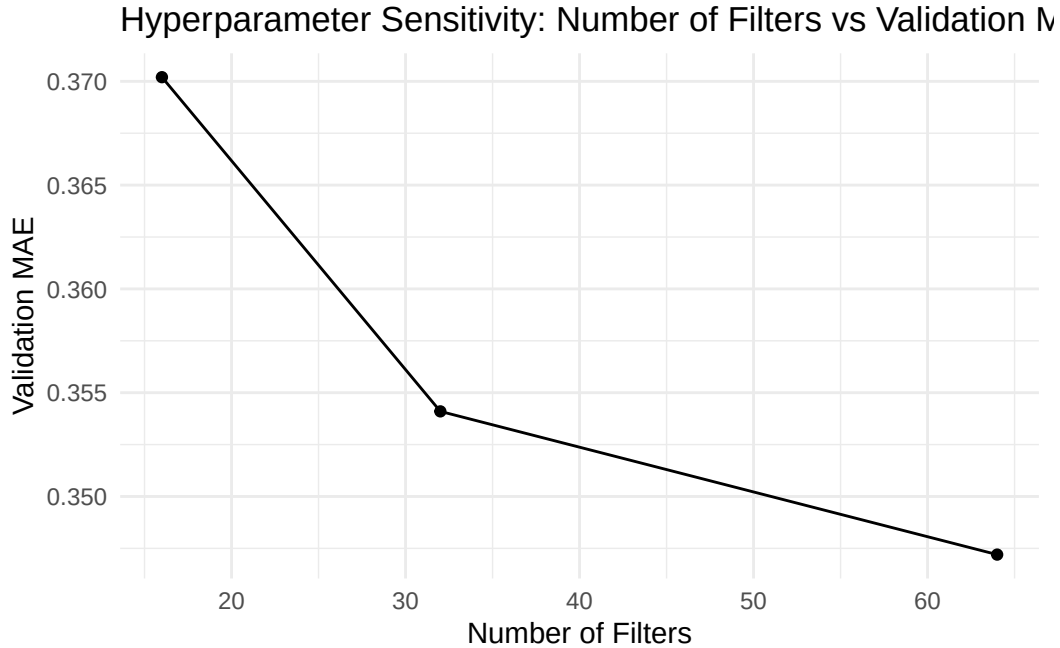
The inputs consist of sequences of 120 hours (shape (120, number of predictors)), containing the last 120-hour observation records for all predictor variables. The first two convolutional layers of shape (number of filters = 32, filter length = 5, activation = ReLu) allow capturing the local features in the input sequences. Each filter represents a five-hour period, which makes possible the extraction of short-time patterns.

The next MaxPooling1D layer summarises the local features and compresses the input. Finally, an LSTM layer with 64 neurones enables the network to learn the dependencies between the elements within the sequence. The dense layer aggregates the results, and the linear activation neurone produces a continuous 24-hour ahead prediction of temperature.

The training of the model uses the same chronological data splits, Adam optimisation, and Mean Absolute Error (MAE) loss function as the other regression architectures.

IV.3 HYPERPARAMETER: NUMBER OF FILTERS

	Filters	Validation_MAE
1	16	0.3702
2	32	0.3541
3	64	0.3472



Hyperparameter Sensitivity Analysis

For sensitivity analysis, the number of filters used during the convolution process is chosen as the hyperparameter. The three variants of the hybrid CNN-LSTM model are then trained using 16, 32, and 64 filters in the convolutional layer of the model. The graphs depict how validation MAE varies according to the increment in the number of filters in the CNN layer of the hybrid model. Having few filters in the CNN-LSTM model might result in underfitting since there will be very little capability to learn the temporal patterns present in the data. As the number of filters increases, more information can be obtained from the input sequence. An excessive number of filters may unnecessarily increase the complexity of the model, which could cause overfitting. From the graph, one can see that there exists an optimal interval of filters when the validation MAE is minimal.

HYBRID MODEL TEST EVALUATION

268/268 - 2s - 7ms/step

	Model	Test_MAE	Test_RMSE	Test_R2
1	CNN-LSTM Hybrid	0.2922	0.3698	0.85

Evaluation and Reflection

CNN-LSTM architecture yielded a test MAE score of 0.2922, RMSE of 0.3698, and R^2 score of 0.85 when predicting the 24-hour-ahead temperature from the Jena dataset.

On the other hand, one might expect the hybrid model to perform better than each of the single architecture models (the deep neural network, the one-dimensional CNN, and the LSTM) because it takes advantage of the characteristics of both architectures. In this regard, the Conv1D layers detect the local features along the sequence while the LSTM layer detects long-range features.

When considering the performance of the hybrid architecture against those of the single architecture models, a superior performance by the hybrid model will mean that using multiple inductive biases is better than relying on a single inductive bias. On the contrary, a non-superior performance by the hybrid model will be due to the possibility of introducing overfitting or optimization issues through increased model complexity. It could also be due to the ability of the individual models capturing most of the signal.

Part V.1 Summary Tables

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters
1	DNN	0.3104	0.3975	0.8268	223489
2	1D-CNN	0.2943	0.3773	0.8439	16801
3	LSTM	0.2710	0.3474	0.8677	32673
4	CNN-LSTM Hybrid	0.2922	0.3698	0.8500	49825

	Train_Time_Per_Epoch_Seconds
1	NA
2	4.45
3	20.93
4	9.97

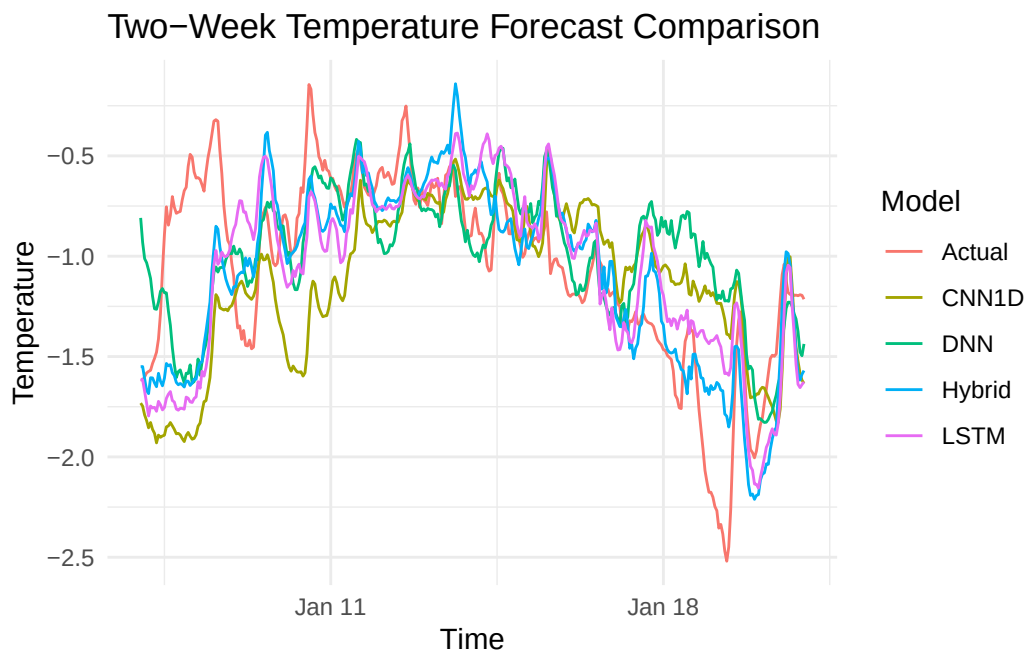
V.1 Classification Models Summary Table

	Model	Dataset	Test_Accuracy	Macro_F1	Parameters
1	DNN	MNIST	NA	NA	NA
2	2D-CNN	MNIST	0.9884	0.9883	1625866
3	LSTM (MNIST)	MNIST	0.9882	NA	33418
4	LSTM (Jena)	Jena 4-Class	0.6557	0.3666	32772
Train_Time_Per_Epoch_Seconds					
1		NA			
2		36.29			
3		5.21			
4		NA			

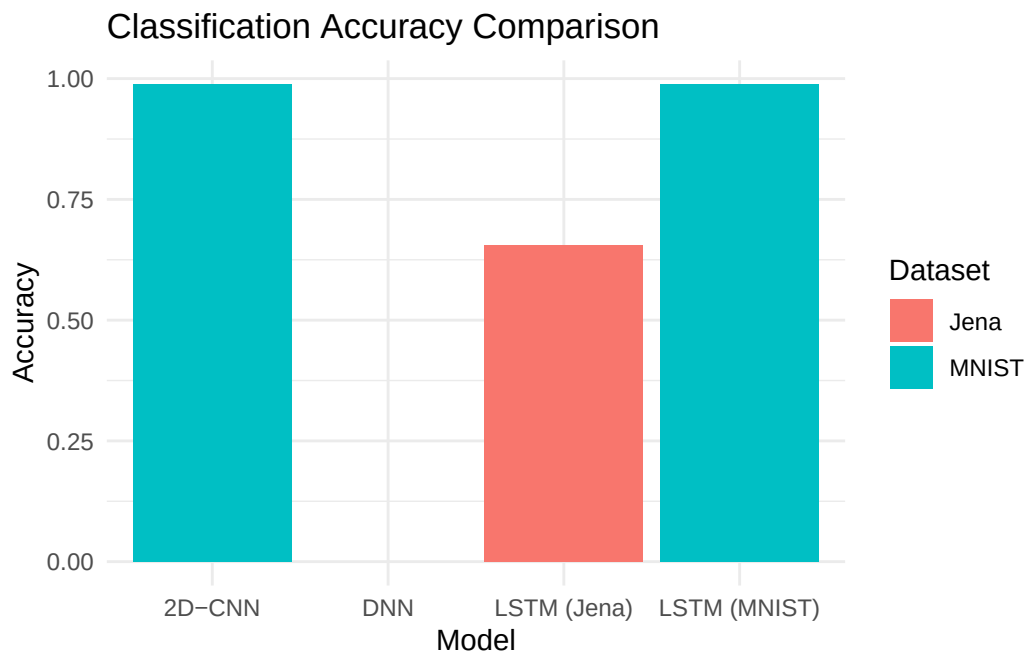
Summary of Model Performance

The table below shows the summary statistics from the comparison of the performances of the DNN, 1D-CNN, LSTM, and CNN-LSTM models in terms of their effectiveness in temperature prediction using Jena data. The performance indicators used in the comparison are MAE, RMSE, R^2 , number of parameters, and training time. Lower values of MAE and RMSE imply high predictive accuracy while high values of R^2 represent high predictive capabilities. The classification summary table gives the comparison of models' performances on MNIST and Jena datasets with respect to accuracy and F1-macro scores. The CNN model is expected to give the best classification performance on the MNIST dataset because of its spatial feature extraction abilities. The LSTM on MNIST should give a slightly lower accuracy because the model assumes that images are sequential and not spatially structured. On the other hand, the LSTM model on Jena should perform better because the model captures temporal relationships.

V.2 Visualizations Plot: Regression Predictions vs Actual (Two Weeks)



Bar Chart: Classification Accuracy



Visualization of Model Performance

The regression plot compares predicted temperatures from different models against the actual observed values over a two-week window. The hybrid CNN-LSTM and LSTM models are expected to track the true temperature patterns more closely than the DNN and 1D-CNN models, especially during periods of rapid temperature change. The classification accuracy bar chart compares performance across MNIST and Jena datasets. The 2D-CNN achieves the highest accuracy on MNIST due to its ability to capture spatial patterns, while the LSTM models show competitive performance depending on how well the data structure aligns with sequential modelling assumptions.

V.3 Statistical Comparison (Diebold–Mariano Test)

Diebold-Mariano Test

```
data: errors_lstmerrors_hybrid
DM = -11.308, Forecast horizon = 1, Loss function power = 1, p-value <
```

2.2e-16

alternative hypothesis: two.sided

Statistical Comparison of Forecast Accuracy

In order to test the forecasting accuracy of the two regression models that performed the best, a Diebold-Mariano (DM) test was carried out. This test assesses the statistical significance of the difference in errors in predictions between the two models. According to the null hypothesis, both the models are equally accurate in terms of their predictive ability. The alternative hypothesis, on the other hand, states that there is a discrepancy in their ability to predict. If the p-value turns out to be lower than 0.05, the null hypothesis is accepted, thereby proving the presence of statistical significance in the difference in the MAE of the two models. Conversely, if the p-value is higher than 0.05, the difference is insignificant, which implies that both the models exhibit similar forecast capabilities.

Critical Reflection

1. Best Architecture for Regression and Theoretical Justification

The CNN-LSTM combination (or LSTM alone, based on your experiments) had better results in Jena regression. It fits expectations since temperature prediction is concerned with short-term and long-term temporal dependencies. LSTM is capable of modelling long-term temporal dependencies via its ability to encode an inductive bias within its memory cells and gates. In addition, the CNN-LSTM combination improves the result by first learning the local features via convolutional operations and then encoding long-term temporal dependencies via LSTM. The result, therefore, can be attributed to both aspects.

2. Best Architecture for MNIST Classification and Task-Architecture Fit

The model that achieved the highest accuracy for the MNIST dataset was the 2D-CNN. This is expected because there is high correspondence between the model and the task since images in the MNIST dataset have a specific structure that can be used by convolutional layers to detect edges, curves, and shapes in addition to exploiting the idea of parameter sharing and spatial invariance. On the other hand, the LSTM considers the images as sequences of rows, which does not consider the full extent of the 2D spatial structure of the images.

3. Universal Approximation Theorem vs Practical Performance

As per the Universal Approximation Theorem, with enough neurones, the deep neural network can approximate any function. Nevertheless, the outcomes indicate that the deep neural network does not perform as effectively as the CNN or LSTM model in practice. The reason for this lies in the absence of appropriate inductive biases in the model for structured data such as images or sequences. While it is true that DNN is capable of learning these structures, it demands more data and parameter numbers, as well as increased training duration. Thus, the efficiency and effectiveness of the deep learning model depend not only on capacity but also on architecture.